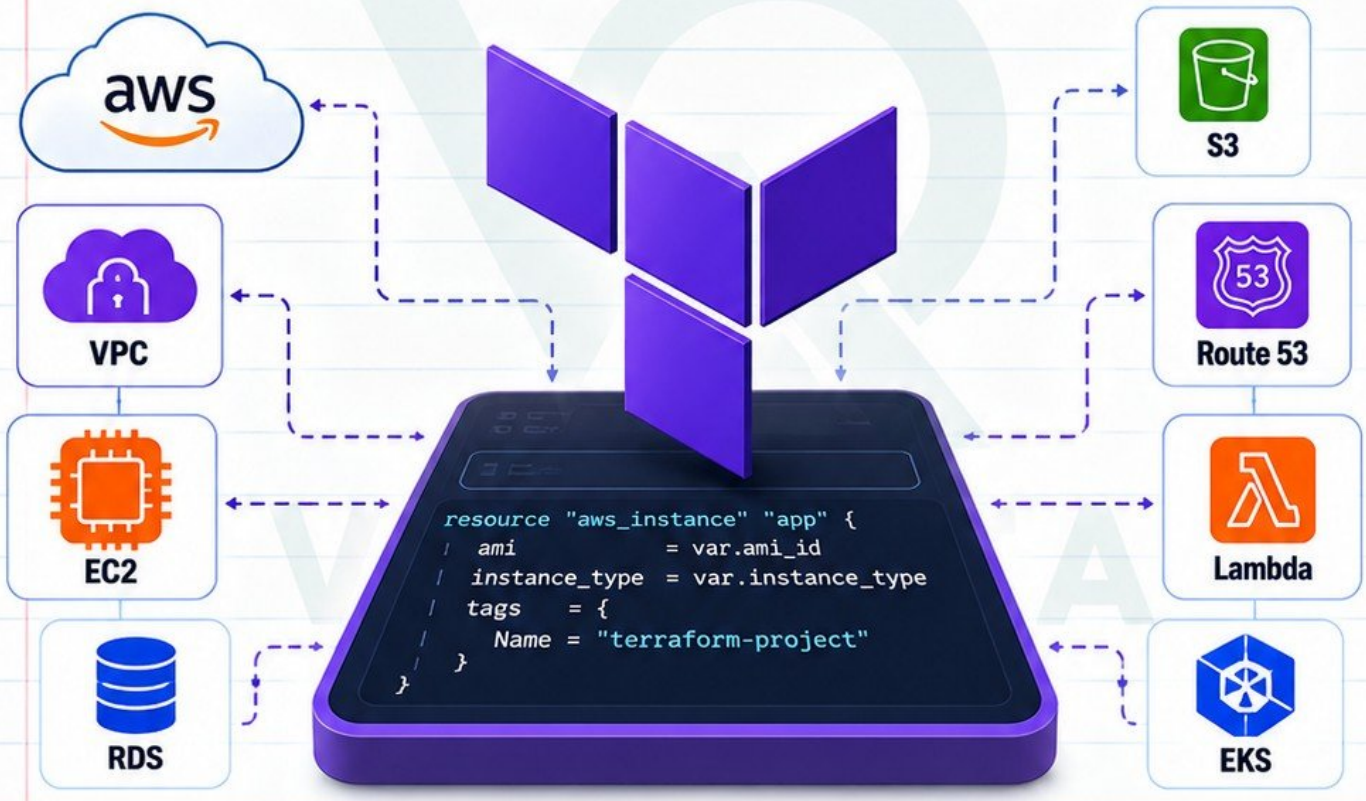
VERIQTA

10 Terraform Projects for Your Portfolio

Build Real-World Infrastructure Projects to Showcase Your DevOps and Cloud Engineering Skills



PRACTICAL PROJECTS | **REAL-WORLD SCENARIOS** | **PRODUCTION-READY CONFIGURATIONS**

@veriqta @veriqta @veriqta @veriqta @veriqta @veriqta

Project 01

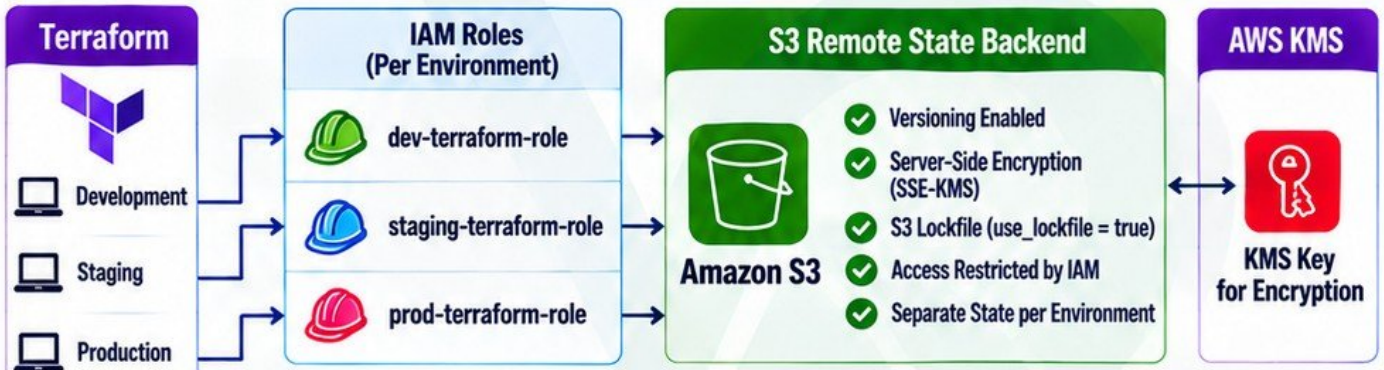
VERIQTA

Build a Terraform Backend That Can Survive Operator Mistakes

Project: Design a secure, recoverable Terraform state management system on AWS.

Build an S3-backed Terraform state architecture with versioning, encryption, IAM isolation, and native S3 lockfile support.

Architecture Overview



Engineering Requirements

1. Configure S3 remote state with use_lockfile = true.
2. Enable S3 versioning and server-side encryption.
3. Restrict state access through workload-specific IAM roles.
4. Separate development and production state boundaries.
5. Prevent unauthorized state deletion.
6. Design a recovery procedure for corrupted or accidentally modified state.

Terraform Backend Configuration (Example)

```
terraform {
  backend "s3" {
    bucket       = "veriqta-terraform-state"
    key          = "production/infra/terraform.tfstate"
    region       = "us-east-1"
    encrypt      = true
    kms_key_id   = "arn:aws:kms:us-east-1:123456789012:key/xxxx"
    use_lockfile = true
  }
}
```

S3 Bucket Configuration (Key Settings)

- Versioning Enabled
- SSE-KMS Encryption
- Block Public Access
- Object Lock (Deletion Protection)
- Lifecycle Policies

IAM Role Design (Least Privilege)

- Separate IAM roles for dev, staging, and production.
- Allow only required S3, DynamoDB (if used), and KMS actions.
- Use least privilege permissions.
- Prevent cross-environment access.
- Enable MFA for sensitive operations.

Architecture Decision

Centralized backend governance versus isolated state and independent recovery.

Portfolio Evidence

- Terraform backend configuration.
- IAM access design.
- State recovery procedure.
- Documented security controls.

Production Incident: Terraform State Was Accidentally Overwritten

Failure scenario: An engineer performs a state operation against the wrong enviropment. Terraform now proposes replacing production infrastructure that is still running.

What Happened?



Investigate and Diagnose

1. Compare actual AWS resources with Terraform state.
2. Inspect S3 object versions and CloudTrail activity.
3. Identify whether the state, configuration, or real infrastructure changed.
4. Determine whether state restoration, import, or configuration reconciliation is appropriate.
5. Avoid applying the destructive plan.

Check S3 State Versions

S3 > Buckets > veriqta-terraform-state > Object versions

Version ID	Last modified	Size	Actions
3HL4kq...(current)	Oct 28, 2026 14:21	24 KB	Download
9Gf7sD...	Oct 28, 2026 12:10	24 KB	Download
1a2b3c...	Oct 27, 2026 09:15	24 KB	Download

Check CloudTrail Activity

```
{
  "eventTime": "2026-10-28T14:21:33Z",
  "eventName": "PutObject",
  "userIdentity": {
    "arn": "arn:aws:sts::123456789012:assumed-role/dev",
    "userName": "dev-user"
  },
  "requestParameters": {
    "bucketName": "veriqta-terraform-state",
    "key": "production/terraform.tfstate"
  }
}
```

Compare Real Infrastructure vs Terraform State

Actual AWS Resources (still running)

- ✓ VPC (vpc-0abc123)
- ✓ EC2 instances (2 running)
- ✓ RDS database (available)
- ✓ ALB (active)
- ✓ All resources healthy

Terraform Plan (after state overwrite)

- ✗ Plan proposes to destroy VPC
- ✗ Plan proposes to replace RDS
- ✗ Plan shows 20+ resources for recreation
- ✗ No actual infrastructure issues
- ✗ Indicates state mismatch

Determine the Right Recovery Approach

Situation	Approach
State file corrupted but infrastructure unchanged	Restore previous S3 object version
Resources exist but not in state	Use terraform import
Configuration changed incorrectly	Revert or fix configuration and re-run plan
Both state and configuration changed	Combine restoration, import, and configuration reconciliation

Senior Challenge

Recover state consistency without recreating production resources.

Verification

A reviewed terraform plan shows no unintended changes.

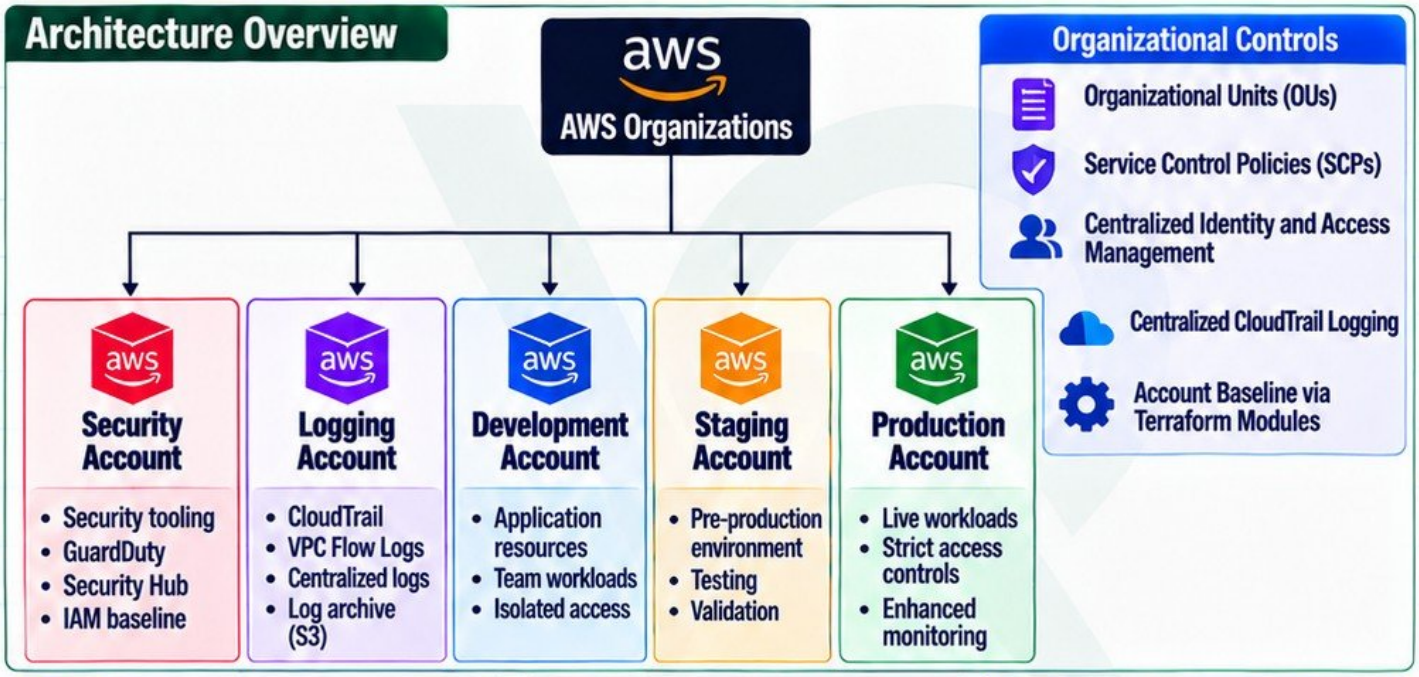
Portfolio Deliverable

- Documented state recovery exercise.
- Before and after terraform plan outputs.
- S3 version restoration evidence.
- CloudTrail investigation screenshots.
- Final configuration and validation results.

Build an AWS Landing Zone With Real Security Boundaries

Project: Provision a controlled multi-account AWS foundation.

Architecture Overview



Organizational Controls

- Organizational Units (OUs)
- Service Control Policies (SCPs)
- Centralized Identity and Access Management
- Centralized CloudTrail Logging
- Account Baseline via Terraform Modules

Key Components

- **AWS Organizations** — Manage multiple AWS accounts.
- **Organizational Units (OUs)** — Group accounts by function.
- **Service Control Policies (SCPs)** — Enforce guardrails and restrictions.
- **Cross-Account IAM Roles** — Allow secure deployments from a central account.
- **Centralized CloudTrail Logging** — Capture all account activity.
- **Terraform Modules** — Automate account-level baseline configuration.

Engineering Decisions

1. Which security controls belong at the organization level?
2. Which permissions should remain with application teams?
3. How should Terraform authenticate across accounts?
4. How can account provisioning remain repeatable?

Terraform Structure (Example)

```
landing-zone/
├── main.tf
├── variables.tf
├── providers.tf
├── modules/
│   ├── organization/
│   ├── accounts/
│   ├── scp/
│   ├── iam-roles/
│   └── logging/
├── environments/
│   ├── dev/
│   ├── staging/
│   └── prod/
└── README.md
```

Senior Insight

AWS accounts reduce certain blast-radius risks, but shared identities, networking, and automation can reintroduce correlated failures.

Portfolio Deliverable

- Multi-account architecture.
- Policy definitions (SCPs and IAM).
- Account bootstrap process.
- Terraform module implementation.
- Access-control evidence.

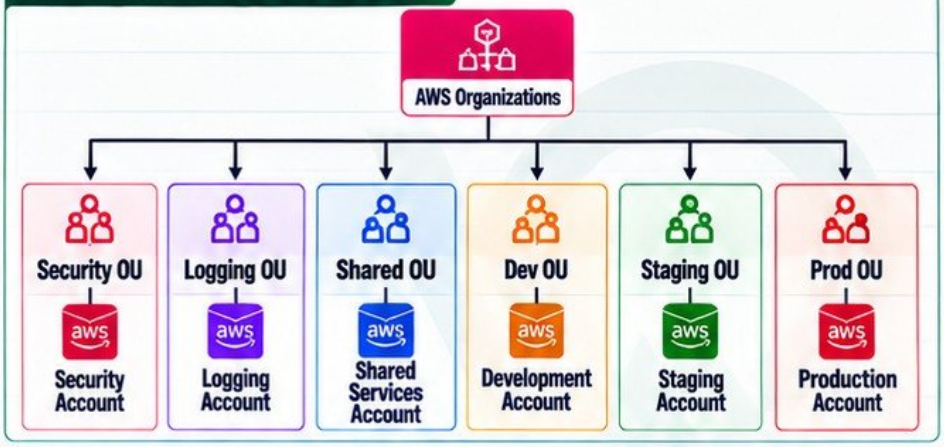
Expected Outcome

A secure, repeatable multi-account AWS foundation with clear security boundaries and auditable access controls.

Production Incident: A Security Policy Disabled Deployments

Failure scenario: A new Service Control Policy prevents production pipelines from updating required AWS resources.

Environment Overview



How the Incident Happened

A new Service Control Policy (SCP) was deployed to an OU.

↓

The SCP accidentally denied required API actions.

↓

Production deployment pipelines began to fail.

Example Error from Pipeline

```
Error: updating EC2 Instance (i-0abc123...):
operation error EC2: ModifyInstanceAttribute,
https response error StatusCode: 403,
RequestID: a1b2c3d4-...,
api error UnauthorizedOperation:
You are not authorized to perform this operation.
This action is explicitly denied by a Service
Control Policy.
```

Investigate and Diagnose

1. Identify the denied API actions.
2. Correlate CloudTrail events with the policy deployment.
3. Distinguish IAM permissions from organization-level restrictions.
4. Test the policy in a controlled account before wider deployment.
5. Design a staged rollout and recovery mechanism.

Check CloudTrail Events

```
{
  "eventTime": "2026-10-28T14:21:33Z",
  "eventSource": "ec2.amazonaws.com",
  "eventName": "ModifyInstanceAttribute",
  "errorCode": "AccessDenied",
  "errorMessage": "User: arn:aws:sts::123456789012:
    assumed-role/deploy-role is not authorized to
    perform this operation. Encoded authorization
    failure message: ..."
}
```

Distinguish IAM vs SCP

Aspect	IAM Policy	Service Control Policy (SCP)
Scope	Specific user/role	Entire account or OU
Purpose	Allow or deny permissions	Set guardrails across accounts
Evaluation	Evaluated with other policies	Acts as an account-level permission boundary
Common Error	Missing permissions	Explicit deny in SCP
Fix	Update IAM policy	Modify or exclude from SCP

Senior Challenge

Restore deployment capability without broadly weakening organizational security.

Verification

Required deployments succeed while prohibited operations remain blocked.

Portfolio Deliverable

- Policy failure analysis.
- Corrected SCP.
- Staged rollout evidence.
- CloudTrail investigation results.

Key Best Practices

- Test SCPs in a sandbox account.
- Use explicit exclusions for required services.
- Roll out policies gradually.
- Monitor with CloudTrail and alerts.

Project 03

VERIQTA

Build a VPC Where Private Workloads Cannot Freely Reach the Internet

Project: Design an AWS VPC with controlled outbound connectivity.

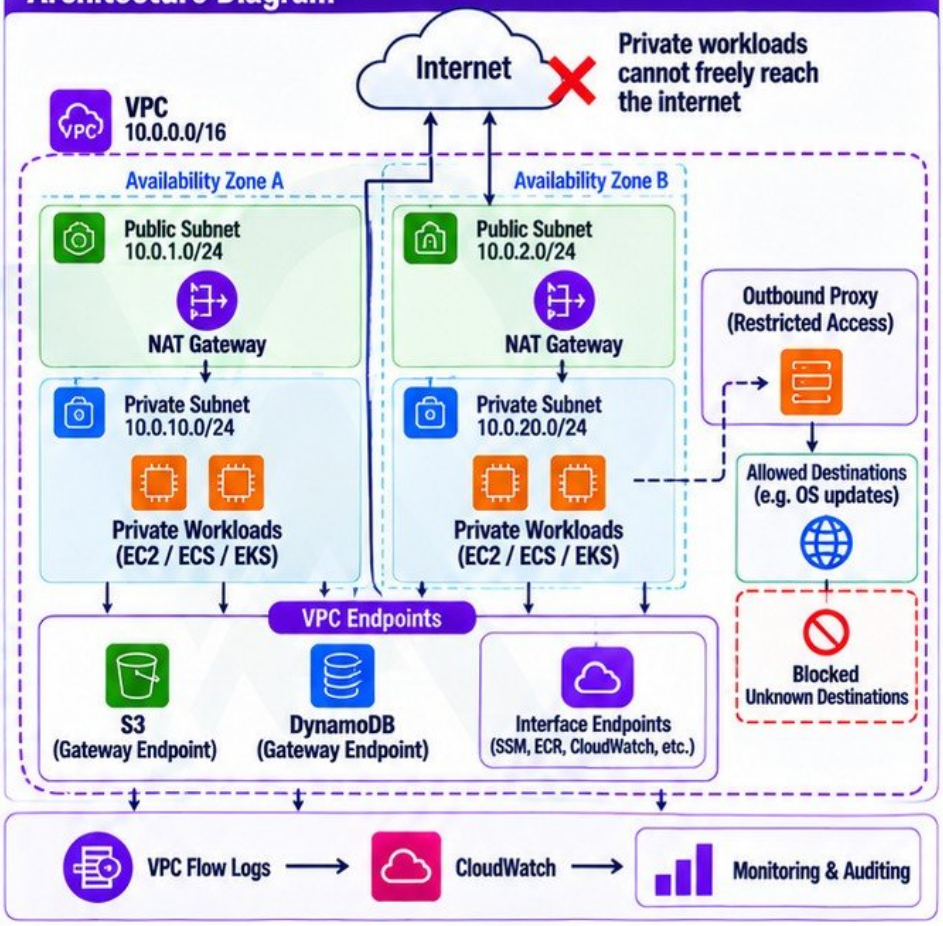
Key Requirements

1. Multi-AZ VPC architecture.
2. Public and private subnets.
3. Route tables and security groups.
4. NAT gateways where general outbound access is required.
5. Gateway endpoints for S3 and DynamoDB.
6. Interface endpoints for selected AWS services.
7. VPC Flow Logs.
8. Restricted outbound proxy for destinations requiring explicit control.

Route Table Design (Example)

Subnet Type	Destination	Target
Public Subnet	0.0.0.0/0	Internet Gateway
Private Subnet (with NAT)	0.0.0.0/0	NAT Gateway
S3/DynamoDB	S3 / DynamoDB Prefixes	Gateway Endpoint
AWS Services	Service Prefixes	Interface Endpoint
Restricted Outbound	Allowed IPs/Domain	Outbound Proxy

Architecture Diagram



Security Controls

- Restrictive security groups (least privilege).
- No direct internet access from private subnets.
- Allow only required outbound traffic.
- Use VPC endpoints with endpoint policies.
- Enable VPC Flow Logs for monitoring.

Terraform Module Structure (Example)

```
vpc/
├── main.tf
├── variables.tf
├── subnets/
├── route-tables/
├── security-groups/
├── nat-gateway/
├── vpc-endpoints/
├── flow-logs/
└── outputs.tf
```

Architecture Trade-off

NAT Gateway (Simplicity)	VS	VPC Endpoints (Cost & Control)
• Simple setup		• No internet access
• Broad internet access		• Lower data transfer cost
• Predictable routing		• More granular control
• Higher ongoing cost		• More components to manage

Portfolio Deliverable

- Terraform network modules.
- Routing diagrams.
- Security rules.
- Connectivity test results.

Expected Outcome

A secure VPC where private workloads can only reach approved destinations through NAT, VPC endpoints, or a restricted outbound proxy.

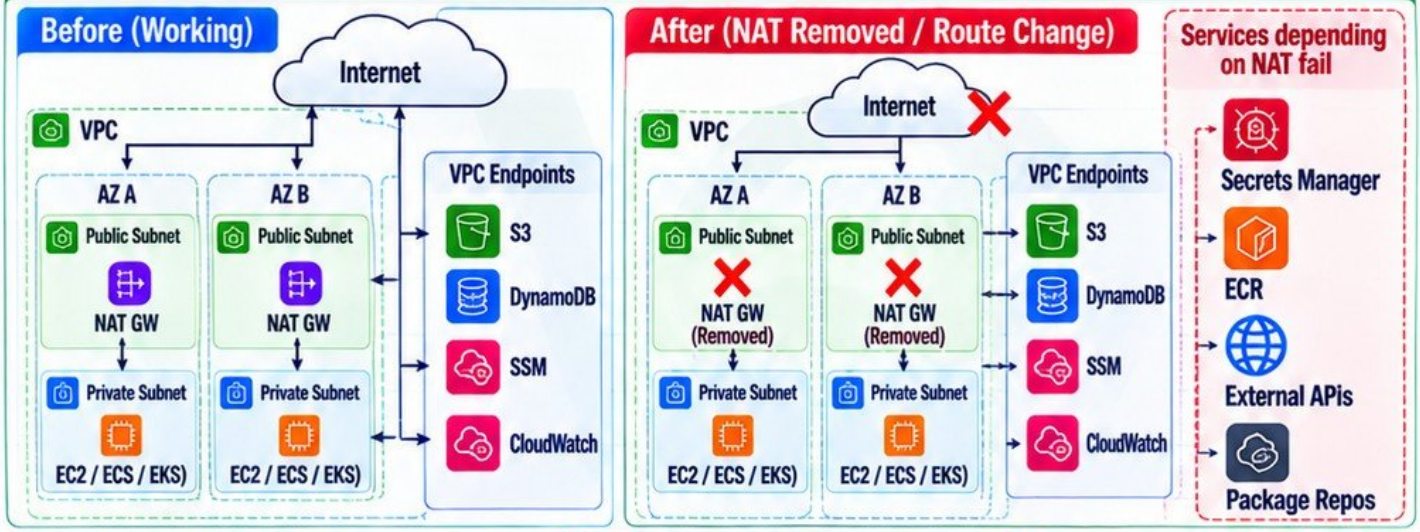
Connectivity Test Examples

```
# Should succeed (AWS service via endpoint)
curl -I https://s3.amazonaus.com
# Should succeed (alowed destination via proxy)
curl -I https://example.com
# Should fail (blocked internet access)
curl -I https://google.com
# Connection timed out
```

Production Incident: Private Applications Lost Access to AWS Services

Failure scenario: A route change removes NAT connectivity. Some applications continue operating while others fail to retrieve secrets or publish logs.

Architecture Before and After (Simplified)



Symptoms and Evidence

Component	Status	Observation
Applications	✗	Some running, some failing
Secrets retrieval	✗	Timeout errors
Log publishing	✗	No logs in CloudWatch
S3 access (via VPC endpoint)	✓	Working
DynamoDB access (via VPC endpoint)	✓	Working
External API access	✗	Connection timed out

Investigate and Diagnose

1. Identify which services use VPC endpoints and which depend on NAT.
2. Inspect route tables, endpoint policies, security groups, and DNS.
3. Correlate VPC Flow Logs with application errors.
4. Determine whether the outage affects one AZ or the entire VPC.
5. Restore only the required connectivity.

Check Route Tables (Example)

Route Table	Destination	Target
Public RT	0.0.0.0/0	Internet Gateway
Private RT (Before)	0.0.0.0/0	NAT Gateway
Private RT (After)	0.0.0.0/0	(Removed)
S3 Endpoint	pl-xxxx	VPC Endpoint
DynamoDB Endpoint	pl-xxxx	VPC Endpoint

VPC Flow Logs (Example)

```
2
2026-10-28T14:21:10Z  REJECT  10.0.10.25  52.95.28.10   443
2026-10-28T14:21:11Z  REJECT  10.0.20.30  3.5.140.10    443
2026-10-28T14:21:12Z  ACCEPT  10.0.10.25  52.216.32.12  443
2026-10-28T14:21:13Z  ACCEPT  10.0.10.25  52.95.110.5   443
```

Application Error (Example)

```
Error: failed to retrieve secret from Secrets Manager
RequestError: send request failed
caused by: dial tcp 52.95.28.10:443: i/o timeout
ERROR [cloudwatch] failed to publish logs:
Post "https://logs.amazonaus.com":
dial tcp 52.95.110.5:443: i/o timeout
```

Senior Challenge

Recover access without introducing unrestricted outbound internet connectivity.

Verification

- Required AWS API calls succeed (e.g. Secrets Manager, CloudWatch).
- Prohibited destinations (e.g. arbitrary internet access) remain inaccessible.

Portfolio Deliverable

- Network failure simulation.
- Diagnostic evidence (Flow Logs, route tables, errors).
- Corrected Terraform configuration.
- Connectivity test results.

Key Takeaway

Use VPC endpoints and controlled egress to allow only required connectivity instead of broad internet access.

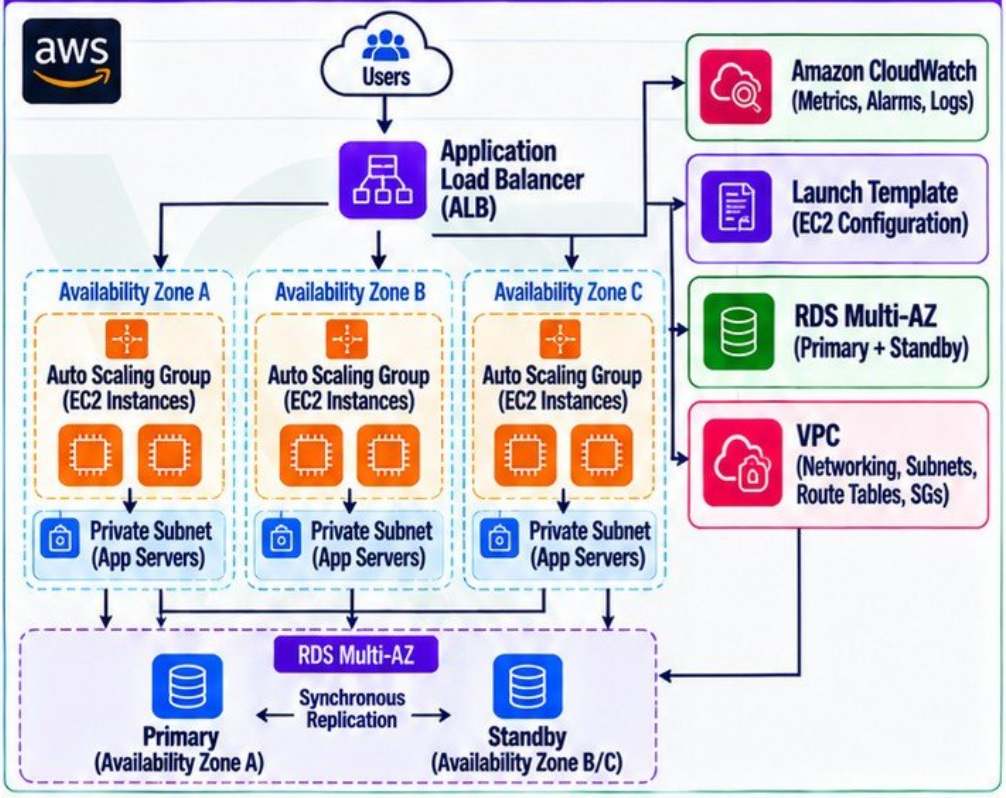
Build a Three-AZ Application That Proves Its Availability

Project: Deploy a production-style application architecture on AWS.

Key Requirements

1. Application Load Balancer.
2. EC2 Auto Scaling across three AZs.
3. Launch templates.
4. RDS Multi-AZ database.
5. CloudWatch metrics and alarms.
6. Health checks.
7. Terraform-managed scaling and network configuration.
8. High availability and fault tolerance design.

Architecture Diagram (Three Availability Zones)



Terraform Structure (Example)

```
three-az-app/
├── main.tf
├── variables.tf
├── vpc/
│   └── main.tf
├── alb/
│   └── main.tf
├── autoscaling/
│   ├── main.tf
│   └── launch-template.tf
├── rds/
│   └── main.tf
├── monitoring/
│   └── main.tf
└── outputs.tf
```

Key Components and Purpose

Component	Purpose
Application Load Balancer	Distribute traffic across three AZs.
EC2 Auto Scaling	Automatically scales instances across three AZs.
Launch Template	Standardized EC2 configuration.
RDS Multi-AZ	High availability database with synchronous replication.
CloudWatch	Metrics, alarms, and health monitoring.
VPC (3 AZs)	Public and private subnets, route tables, security groups.
Health Checks	Ensure unhealthy instances are replaced.
Terraform	Infrastructure as code for consistent deployments.

Capacity Planning Question

How much spare capacity is required to survive losing one AZ?

- Calculate required capacity per AZ.
- Evaluate if two remaining AZs can handle the full workload.
- Consider CPU, memory, latency, and database connections.
- Test with load simulation.

CloudWatch Metrics (Examples)

- ALB RequestCount / TargetResponseTime
- EC2 CPUUtilization / StatusCheckFailed
- Auto Scaling Group Desired vs InService
- RDS CPUUtilization / FreeableMemory
- Application errors and latency

Architecture Decision

Evaluate whether two remaining AZs can absorb the workload without exceeding latency or saturation limits.

Portfolio Deliverable

- Infrastructure modules.
- Capacity assumptions.
- Load-testing results.
- Availability design.

Expected Outcome

A highly available application that remains operational when one AZ fails.

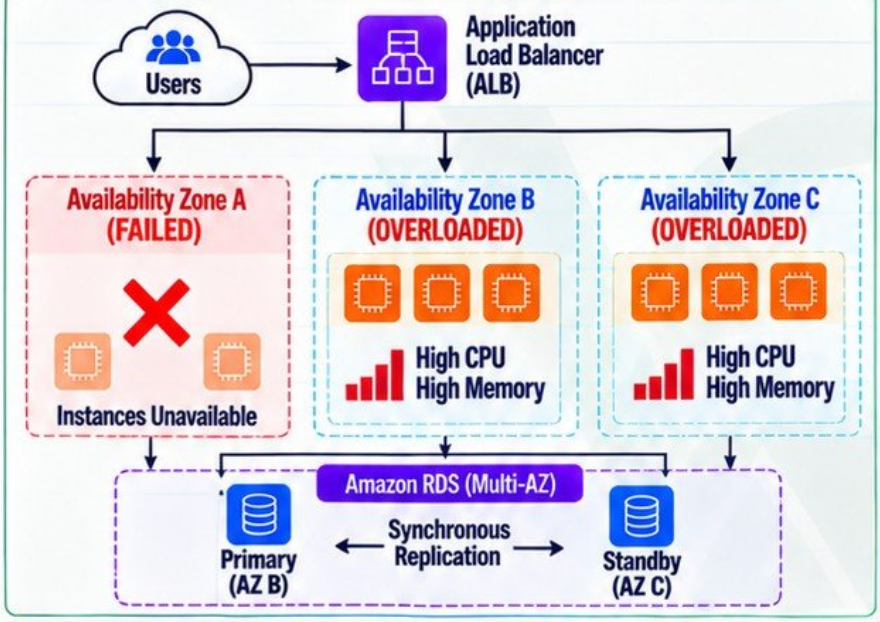
Testing and Validation

- Simulate an AZ failure.
- Verify automatic recovery.
- Check application latency.
- Confirm no single point of failure.

Production Incident: One AZ Failed and the Remaining Two Collapsd

Failure scenario: An AZ outage causes the load balancer to redirect traffic. The remaining instances become saturated, and database connections increase sharply.

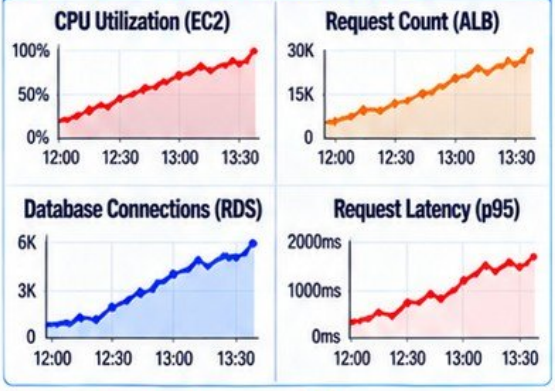
Architecture During AZ Failure (Simplified)



What Happened

1. One Availability Zone became unavailable.
2. ALB redirected traffic to the remaining two AZs.
3. EC2 instances became saturated (high CPU and memory).
4. Database connections increased sharply.
5. Request latency and error rates increased.

CloudWatch Metrics (Example)



Investigate and Diagnose

1. Target health and traffic distribution.
2. Auto Scaling activity and launch failures.
3. Database connection utilization.
4. CPU, memory, latency, and request errors.
5. Whether scaling capacity was available before the failure.

Example ALB Target Health

AZ	Target Status	Health Check
us-east-1a	Unhealthy	Failed
us-east-1b	Healthy	Healthy
us-east-1c	Healthy	Healthy

Auto Scaling Activity (Example)

```
2026-10-28 13:02:11  Launching a new EC2 instance
2026-10-28 13:02:45  Launch failed (Insufficient
                     capacity in us-east-1c)
2026-10-28 13:03:12  Attempting in us-east-1b
2026-10-28 13:03:40  Successfully launched
                     instance i-0abc123...
```

Senior Challenge

Improve resilience without blindly doubling infrastructure spending.

Verification

Repeat the failure exercise and demonstrate that the remaining capacity meets the defined service-level objective.

Portfolio Deliverable

- Failure test report.
- Capacity calculations.
- Terraform improvements.
- Updated architecture design.

Key Improvements

- Increase capacity buffer.
- Use mixed instance types.
- Implement predictive scaling.
- Optimize database connection management.
- Refine health checks and AZ balancing.

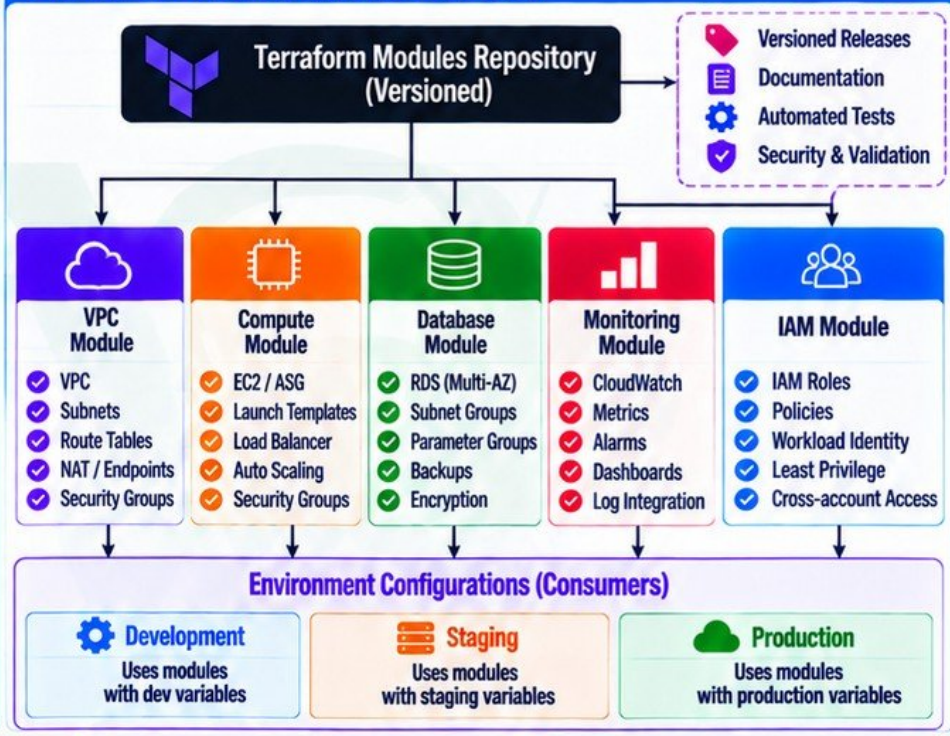
Build an Internal Infrastructure Platform That Developers Can Actually Use

Project: Create reusable Terraform modules for application infrastructure.

Key Requirements

1. VPC networking module.
2. Application compute module.
3. Database module.
4. Monitoring module.
5. IAM workload identity module.
6. Environment-specific configurations.
7. Automated validation and security checks.
8. Versioned module releases.
9. Input validation.
10. Outputs with clear contracts.
11. Deprecation policies.
12. Module compatibility testing.

Platform Architecture (Module Structure)



Example Module Structure

```
terraform-modules/
├── vpc/
│   ├── main.tf
│   ├── variables.tf
│   ├── outputs.tf
│   └── README.md
├── compute/
│   ├── main.tf
│   ├── variables.tf
│   ├── outputs.tf
│   └── README.md
├── database/
│   ├── main.tf
│   ├── variables.tf
│   ├── outputs.tf
│   └── README.md
├── monitoring/
│   ├── main.tf
│   ├── variables.tf
│   ├── outputs.tf
│   └── README.md
├── iam/
│   ├── main.tf
│   ├── variables.tf
│   ├── outputs.tf
│   └── README.md
└── examples/
    ├── dev/
    ├── staging/
    ├── prod/
    └── .github/
        workflows/
```

Module Usage Example (Consumer)

```hcl
module "vpc" {
  source = "github.com/veriqta/terraform-
  modules//vpc?ref=v1.0.0"
  name     = "app-vpc"
  cidr_block  = "10.0.0.0/16"
  environment = "dev"
}
```

Input Validation Example

```hcl
variable "cidr_block" {
  description = "VPC CIDR block"
  type        = string
  validation {
    condition = can(cidrhost(var.cidr_block, 0))
    error_message = "Invalid CIDR block format."
  }
}
```

Module Outputs (Contract)

```hcl
output "vpc_id" {
  description = "VPC ID"
  value = aws_vpc.main.id
}
output "private_subnet_ids" {
  description = "Private subnet IDs"
  value  = aws_subnet.private[*].id
}
```

Versioning and Deprecation

v1.0.0 Stable → **v1.1.0** New Features → **v2.0.0** (Deprecate Old Inputs)

- Maintain backward compatibility where possible.
- Document deprecations clearly.
- Provide migration guidance.
- Use semantic versioning (MAJOR.MINOR.PATCH).

Senior Insight

A reusable module is a platform API. Breaking its interface can disrupt dozens of teams.

Portfolio Deliverable

- Published module repository.
- Example consumers.
- Versioning policy.
- Automated test evidence.

Verification

- Modules deploy successfully in multiple environments.
- Validation and security checks pass.
- Compatibility tests succeed.

Key Best Practices

- Design clear, stable interfaces.
- Write good documentations.
- Automate testing and validation.
- Follow semantic versioning.
- Plan deprecations early.

Production Incident: A Module Upgrade Plans to Replace 40 Databases

Failure scenario: A shared database module changes a resource identity. Consumer pipelines begin proposing database replacement.

Key Investigation Areas

1. Terraform resource addresses and state bindings.
2. Module version changes.
3. terraform plan replacement reasons.
4. Safe state migrations using moved blocks where appropriate.
5. Compatibility between old and new module interfaces.
6. Impact across all consumer environments.
7. Validate plans before merging.

Problem Overview

Before (v1.0.0)

module.database
aws_db_instance.main

40 databases (in production)

Module Upgrade (v2.0.0)

Resource identity changed

After (v2.0.0)

module.database
aws_db_instance.this

Terraform plans to REPLACE 40 databases

Common Replacement Reasons

- Resource address changed.
- Resource type or name changed.
- ForceNew attribute modified.
- Lifecycle rules changed.
- Missing or incorrect state migration.
- Module input structure changed.

Terraform Plan Output (Example)

```
# module.database.aws_db_instance.this
-/+ resource "aws_db_instance" "this" {
    id              = "db-abc123"
    identifier      = "prod-db"
    # (forces replacement)
  ~ engine_version = "13.11" -> "15.2"  # forces replacement
  ~ instance_class = "db.r5.large" -> "db.r6g.large"
    # ...
}
Plan: 40 to add, 0 to change, 40 to destroy.
```

Check Resource Addresses

```
# v1.0.0
module.database.aws_db_instance.main

# v2.0.0
module.database.aws_db_instance.this

# Check state
terraform state list | grep database
```

Safe Migration with Moved Block

```
moved {
  from = module.database.aws_db_instance.main
  to   = module.database.aws_db_instance.this
}
```

- Preserves existing resources.
- Updates state addresses.
- Avoids unnecessary replacement.
- Use for non-breaking refactors.

Module Version Comparison

v1.0.0 (Old)

```
module "database" {
  source = "git::.../db//v1.0.0"
  name   = "prod-db"
  engine = "postgres"
  instance_class = "db.r5.large"
}
```

v2.0.0 (New)

```
module "database" {
  source = "git::.../db//v2.0.0"
  name       = "prod-db"
  engine     = "postgres"
  instance_class = "db.r6g.large"
  # new structure and default values
}
```

Investigation and Resolution Steps

1. Compare module versions and changelog.
2. Review terraform plan replacement reasons.
3. Analyze state and resource addresses.
4. Add moved blocks or state migration as needed.
5. Test in non-production environments.
6. Validate representative consumer pipelines.
7. Roll out with clear migration guide and communication.

Migration Test Results

After adding moved blocks and updating configuration:

```
No changes. Infrastructure
matches the configuration.

Plan: 0 to add,
0 to change,
0 to destroy.
```

Senior Challenge

Deliver the module upgrade without destroying or unnecessarily replacing databases.

Pertfolio Deliverable

- Module migration guide.
- Compatibility tests.
- Before-and-after plans.
- Versioning strategy.
- Updated module repository.

Verification

- No unintended replacements in representative environments.
- Pipelines run successfully.
- Existing databases remain intact.
- Module upgrade is documented and tested.

Key Takeaways

- Stable module interfaces matter.
- Use moved blocks for refactors.
- Always test module upgrades.
- Maintain strong versioning and documentation.
- Communicate changes early.

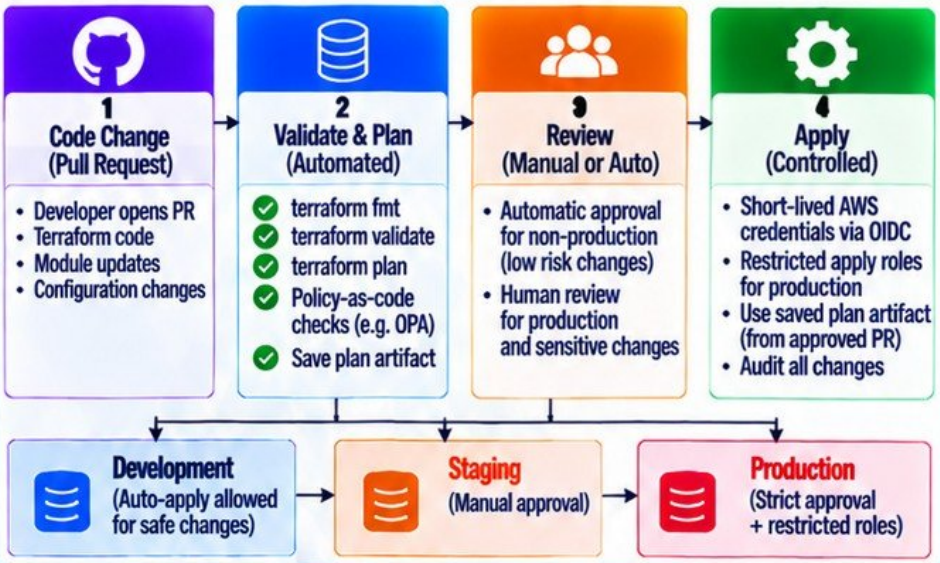
Build a Terraform Delivery Pipeline That Cannot Silently Change Production

Project: Create a controlled infrastructure delivery workflow using GitHub Actions and Terraform.

Key Requirements

1. Pull-request validation.
2. terraform fmt, validate, and plan.
3. Saved plan artifacts.
4. Short-lived AWS credentials through OIDC.
5. Policy-as-code checks.
6. Environment approvals.
7. Restricted production apply roles.
8. Audit trails for infrastructure changes.

Terraform Delivery Pipeline (High-Level Flow)



GitHub Actions Workflow Example

```yaml
name: terrafoom-ci-cd
on:
  pull_request:
    branches: [main]

jobs:
  validate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: hashicorp/setup-terraform@v3
      - run: terraform fmt -check
      - run: terraform validate

  plan:
    needs: validate
    runs-on: ubuntu-latest
    steps:
      - run: terraform plan -out=tfplán
      - uses: actions/upload-artifact@v4
        with:
          name: tfplan
          path: tfplan
```

AWS Authentication with OIDC (Short-Lived Credentials)



Policy-as-Code Checks (Example)

- Check for prohibited resource types.
- Enforce tagging requirements.
- Restrict changes in production.
- Validate security best practices.
- Block public resources.
- Custom organization policies.

```
# Example OPA policy (Rego)
package terraform

deny[msg] {
  resource.type == "aws_s3_bucket"
  not resource.tags["Environment"]
  msg = "S3 bucket must have Environment tag"
}
```

Approval Strategy

Change Type	Environment	Approval Method
Non-sensitive (e.g. tags, alarms)	Development	Automatic
Standard changes	Staging	Manual review
High-risk changes (e.g. network, IAM)	Production	Manual approval (required)

Architecture Decision

Which changes can be automatically approved, and which require human review?

Senior Insight

A reviewed plan must be tied to the exact configuration, provider selections, state context, and authorized execution.

Portfolio Deliverable

- CI/CD workflows.
- Permission boundaries.
- Policy tests.
- Deployment audit evidence.

Verification

Approved changes deploy successfully with full audit trails and no unauthorized or unexpected changes.

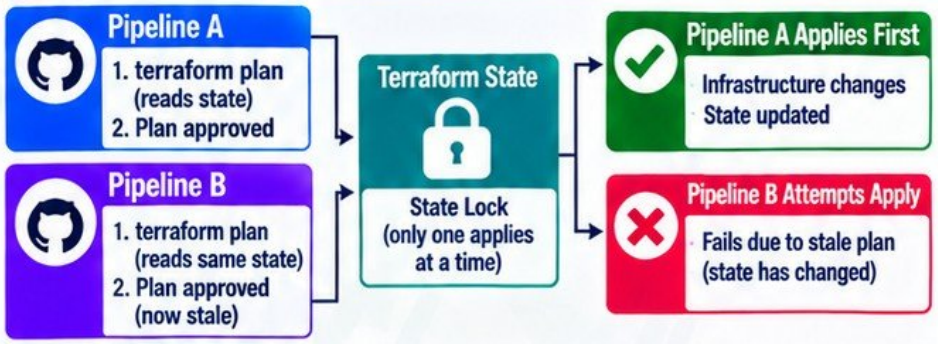
Production Incident: A Terraform Plan Was Approved, but the Infrastructure Changed Before Apply

Failure scenario: Two pipelines plan changes against the same production state. One applies first, making the second plan stale.

Key Investigation Areas

1. State locking behavior.
2. Saved plan validity.
3. Concurrent workflow execution.
4. State serial changes.
5. The difference between planning and applying current infrastructure changes.

Incident Flow (Simplified)



Result: Infrastructure changed before the second apply.

Example: Stale Plan Difference

Planned (Pipeline B)

```
resource "aws_instance" "app" {
  instance_type = "t3.medium"
  ...
}

# Plan shows change from t3.small
# to t3.medium
```

Actual (After Pipeline A Applied)

```
resource "aws_instance" "app" {
  instance_type = "t3.large"
  ...
}

# Infrastructure changed to t3.large
# (not what the second plan expects)
```

What Happens

1. Pipeline A applies changes successfully.
2. State is updated.
3. Pipeline B tries to apply a stale plan.
4. Terraform detects mismatch and fails (or worse, applies unintended changes if not protected).

Terraform Concurrency Controls

- **State Locking** — Prevents multiple applies at the same time.
- **Plan Validation** — Checks if the plan is still valid before apply.
- **Workflow Control** — Serialize or reject concurrent runs in CI/CD.
- **Access Control** — Restrict who can apply to production.

Safe Pipeline Flow (Suggested)



Senior Challenge

Prevent conflicting production applies without granting pipelines unrestricted access.

Verification

- Conflicting workflows are serialized or rejected.
- Stale plans cannot be applied.
- State remains consistent.

Portfolio Deliverable

- Concurrency test results.
- Pipeline controls.
- Documented recovery process.

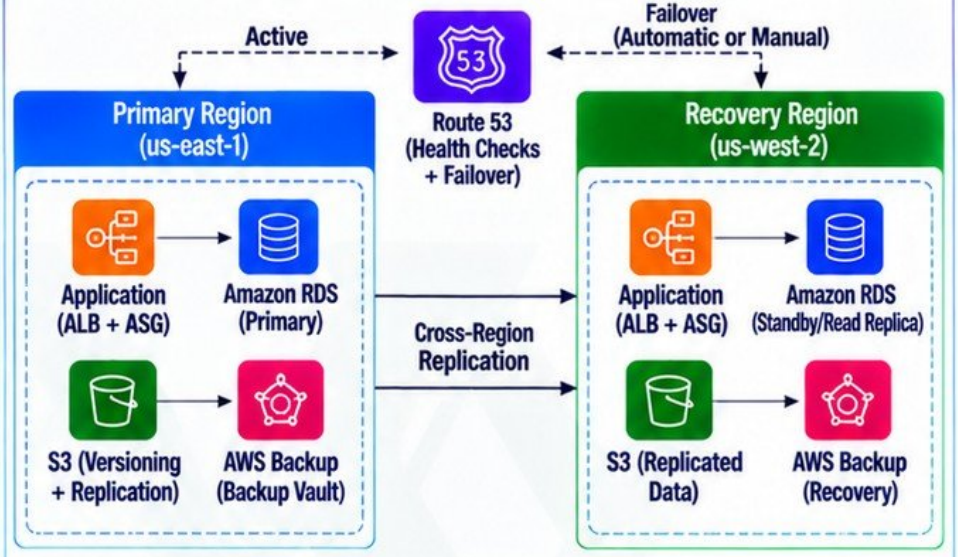
Build a Cross-Region Recovery Environment With Terraform

Project: Design a disaster recovery architecture for a business-critical application.

Key Implementation Areas

1. Primary and recovery AWS Regions.
2. Region-specific Terraform provider configurations.
3. Backup and replication configuration.
4. Recovery networking.
5. IAM and KMS dependencies.
6. DNS failover design.
7. Infrastructure restoration automation.
8. Monitoring and testing.

Cross-Region Disaster Recovery Architecture (Overview)



Terraform Provider Configuration (Multi-Region)

```
provider "aws" {
  region = "us-east-1"
  alias  = "primary"
}

provider "aws" {
  region = "us-west-2"
  alias  = "dr"
```

Backup and Replication

- S3 cross-region replication (with versioning).
- RDS automated backups and cross-region replicas.
- AWS Backup with cross-region backup vault.
- EBS snapshots (cross-region).
- EFS replication (if required).

Recovery Networking

- VPC, subnets and route tables in the recovery region.
- Security groups and NACLs.
- VPC endpoints.
- Transit Gateway or peering (if required).

Key Recovery Objectives

- **Recovery Time Objective (RTO)** Maximum acceptable downtime.
- **Recovery Point Objective (RPO)** Maximum acceptable data loss.
- Data consistency requirements. (e.g. eventual or strong consistency)
- Failover authorization. (who can trigger and how)

Failover Process (High Level)

1. Detect issue and validate impact.
2. Trigger DNS failover (Route 53).
3. Restore or promote database in DR region.
4. Deploy or scale application infrastructure.
5. Verify application health and data consistency.
6. Communicate status and complete switchover.

Architecture Trade-offs

Warm Standby

- Faster recovery (lower RTO).
- Infrastructure always running.
- Higher cost.
- Suitable for critical applications.

Backup and Restore

- Lower cost.
- Longer recovery time (higher RTO).
- Infrastructure created during recovery.
- Suitable for less critical workloads.

Senior Insight

Disaster recovery is not just backup. It requires automated, tested, and measurable recovery capability with clear business requirements.

Portfolio Deliverable

- DR architecture diagram.
- Recovery Terrafrom configurations.
- Backup and replication setup.
- DNS failover design.
- Cost comparison (warm standby vs backup and restore).

Verification

- Failover test completed successfully.
- Application runs in the recovery region.
- Data restored within defined RPO.
- Recovery completed within defined RTO.

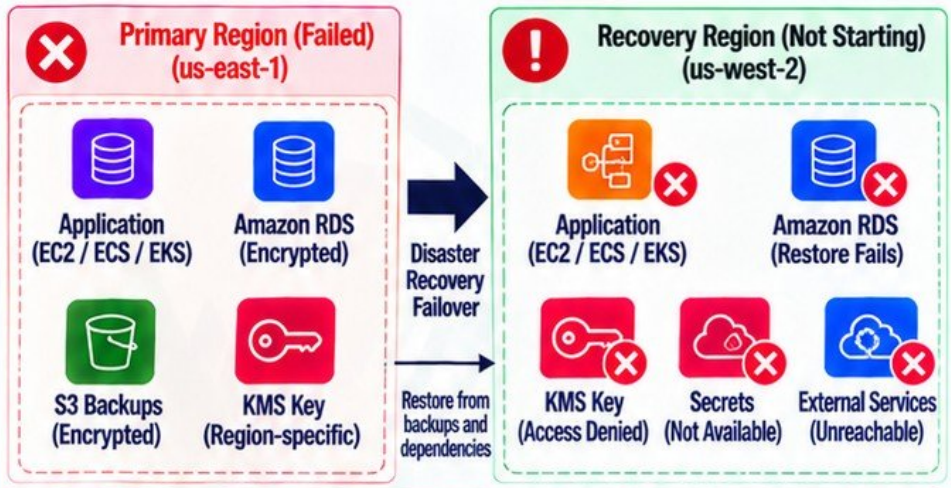
Production Incident: Backups Exist, but the Recovery Region Cannot Start

Failure scenario: The primary Region becomes unavailable. Database backups exist, but the recovery environment cannot access required encryption keys and configuration.

Key Investigation Areas

1. Backup availability and restore permissions.
2. KMS key accessibility and regional dependencies.
3. Secrets, network routing, and external services.
4. Application startup dependencies.
5. Recovery duration and restored data integrity.
6. Logs and error messages across services.
7. Identify services still dependent on the failed Region.
8. Validate actual RTO and RPO performance.

Incident Architecture Overview



Result: Recovery environment cannot start due to missing key access and configuration.

Common Root Causes

- KMS keys not replicated or not accessible in recovery Region.
- Missing permissions for restore and decryption.
- Secrets and configuration stored only in primary Region.
- Network routing, endpoints, or DNS not configured.
- External service dependencies unavailable.
- Application startup dependencies failing (e.g. license servers, APIs).

Recovery Steps (High Level)

1. Verify backup availability and integrity.
2. Ensure KMS keys are accessible in the recovery Region.
3. Restore database from backup.
4. Recreate required secrets and configuration.
5. Configure networking, endpoints and security groups.
6. Deploy or scale application infrastructure.
7. Validate application startup and dependencies.
8. Test data integrity and application functionality.

KMS and Backup Considerations

- Use multi-Region KMS keys or replicate keys to recovery Region.
- Ensure backup encryption keys are accessible.
- Grant required IAM permissions for restore and decryption.
- Verify key policies, grants and cross-account access.
- Test restore process regularly in the recovery Region.

RTO and RPO Validation

- Measure actual Recovery Time Objective (RTO).
- Verify Recovery Point Objective (RPO) and data consistency.
- Document results and compare with business requirements.

Senior Challenge

Restore the application without depending on services unavailable in the failed Region.

Portfolio Deliverable

- Timestamped recovery test.
- Dependency map.
- Failure remediation report.
- Recovery Terraform configurations.

Verification

- Application runs in recovery Region.
- RTO and RPO targets met.
- Data integrity confirmed.
- No dependency on services in failed Region.

Build a Drift Detection System That Knows When Not to Auto-Fix

Project: Create a controlled Terraform drift detection workflow.

Key Implementation Areas

1. Scheduled read-only drift checks.
2. Terraform plan JSON analysis.
3. Classification of changed resources.
4. CloudTrail correlation.
5. Notifications for unexpected changes.
6. Explicit approval before remediation.
7. Separate handling for critical infrastructure.

Drift Detection Workflow (Overview)

1 Scheduled Drift Check (Read-Only)
- Run terraform plan -refresh-only
- Generate JSON output
- No changes applied

2 Analyze Plan Output
- Parse plan JSON
- Identify changed resources
- Classify change type

3 Correlate With CloudTrail
- Check recent API activity
- Identify who made the change
- Verify if change was authorized

4 Notify and Require Approval
- Send detailed notification
- Include classification and evidence
- Require explicit approval before remediation

Example: Terraform Drift Check (Read-Only)

```
# Run a read-only drift check
terraform plan -refresh-only -out=drift.tfplan

# Convert to JSON for analysis
terraform show -json drift.tfplan > drift.json
```

Example: Plan JSON Analysis (Simplified)

```
{
  "resource_changes": [
    {
      "address": "aws_instance.app",
      "change": {
        "actions": ["update"],
        "before": { ... },
        "after": { ... }
      }
    }
] }
```

Drift Classification

Authorized Emergency Intervention
- Made during an incident
- Tracked in change records
- May be kept or re-imported into Terraform

Unauthorized Manual Modification
- No approved change
- Investigate source
- Revert or remediate after approval

External Controller-Managed Change
- Made by AWS services (e.g. Auto Scaling, EKS)
- Expected behavior
- Usually not remediated by Terraform

Provider or Configuration Difference
- Caused by provider version or settings
- Review and update configuration
- May require state update or provider change

Notification Example

Drift Detected

Resource: aws_security_group.app
Change Type: update
Classification: Unauthorized modification
Detected At: 2026-10-28 12:14 UTC
User: arn:aws:iam::123456789012:user/admin
Actions: Review required

Requires Explicit Approval Before Remediation

Remediation Flow (With Approval)

Review Classification and Evidence → Get Explicit Approval → Apply Remediation (If Required) → Verify State Is Consistent

Senior Insight

Not every detected difference should trigger an automatic Terraform apply.

Portfolio Deliverable

- Drift classification workflow.
- Example findings.
- CloudTrail correlation evidence.
- Remediation policy.

Production Incident: Terraform Wants to Undo an Emergency Security Fix

Failure scenario: During an incident, an engineer manually restricts a security group. A scheduled Terraform workflow detects drift and proposes restoring the previous permissive configuration.

Key Investigation Areas

1. CloudTrail evidence of the manual change.
2. Whether the change was authorized.
3. Security implications of the desired Terraform state.
4. Whether configuration or live infrastructure represents the correct policy.
5. How to reconcile the emergency change into code.

Incident Timeline (Simplified)

1. **Security Incident** — Engineer manually restricts a security group to mitigate an active threat.
2. **Scheduled Terraform Run** — Workflow detects drift and proposes to restore the previous permissive configuration.
3. **Potential Impact** — If applied, the workflow would undo the security fix and re-open unnecessary access.
4. **Resolution** — Investigate, validate, approve, and reconcile the emergency change into Terraform configuration.

Result: Emergency security control remains effective and is properly reconciled in code.

Example: CloudTrail Evidence

```
{
  "eventTime": "2026-10-28T14:22:35Z",
  "eventName":"AuthorizeSecurityGroupIngress",
  "userIdentity": {
    "type":  "IAMUser",
    "userName": "oncall-engineer"
  },
  "requestParameters": {
    "groupId": "sg-0abc123def456",
    "ipPermissions": [ ... ]
  }
}
```

Terraform Plan Output (Drift Detected)

```
# aws_security_group.app
~ resource "aws_security_group" "app" {
  ~ ingress = [
    - {
        from_port   = 0
        to_port     = 0
        cidr_blocks = ["0.0.0.0/0"]
      }
    + {
        from_port   = 443
        to_port     = 443
        cidr_blocks = ["10.0.0.0/16"]
      }
  ]
}
```

Security Group Change Comparison

Previous (Permissive - in Terraform State)
Allow inbound from anywhere (0.0.0.0/0) on port 443
(Risk: over-permissive)

Current (Emergency Fix - in Live Infrastructure)
Allow inbound only from 10.0.0.0/16 on port 443
(Improved security)

Incident Investigation Steps

1. Check CloudTrail for who made the change and when.
2. Verify if the change was an authorized incident response.
3. Review the security implications of the Terraform state.
4. Determine which configuration is correct.
5. Document the decision and plan the reconciliation.

Reconciliation Process

1. Update Terraform configuration with the approved change.
2. Run terraform plan to confirm no unintended changes.
3. Submit for review and approval through the normal workflow.
4. Apply and monitor.

Senior Challenge

Prevent automated infrastructure reconciliation from reversing an active security mitigation.

Verification

- Emergency control remains effective.
- Terraform configuration is updated.
- Future plans show no drift.
- Change is reviewed and approved.

Portfolio Deliverable

- Drift incident report.
- Approval workflow.
- Corrected infrastructure code.
- Evidence of investigation and resolution.

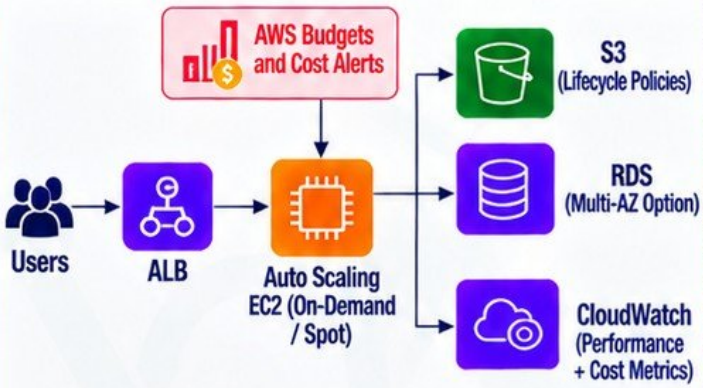
Build Infrastructure That Exposes the Cost of Every Architecture Decision

Project: Create an AWS deployment that measures infrastructure cost alongside reliability and performance.

Key Implementation Areas

1. Terraform resource tagging standards.
2. AWS Budgets.
3. Cost allocation reporting.
4. Cost estimates during pull requests.
5. EC2 capacity configuration.
6. Storage lifecycle policies.
7. Environment-specific infrastructure sizing.
8. CloudWatch performance metrics.

Architecture Overview (Cost-Aware Deployment)



Cost Estimation in Pull Requests



Cost Allocation Report (Example)

Service	Environment	Monthly Cost	% of Total
EC2	production	$320.45	42%
RDS	production	$210.10	27%
S3	production	$45.30	6%
NAT Gateway	production	$120.00	16%
CloudWatch	production	$35.10	5%
Others	production	$30.20	4%
Total	-	**$761.15**	**100%**

CloudWatch Metrics (Performance)

- CPU Utilization
- Memory Utilization
- Request Latency
- Error Rate (5XX/4XX)
- Throughput
- Infrastructure Cost Metrics

NAT Gateway vs VPC Endpoint Costs

NAT Gateway (Internet Access)
Higher cost (e.g. $0.045/hour + data processing)

VPC Endpoint (Private Access)
Lower cost (for AWS services) + more secure

On-Demand vs Spot Instances

On-Demand
- Higher cost
- No interruptions
- Suitable for critical workloads

Spot Instances
- Up to 70% lower cost
- Can be interrupted
- Suitable for interruption-tolerant workloads

Multi-AZ vs Single-AZ Cost

Single-AZ
- Lower cost
- Lower resilience
- Suitable for non-critical workloads

Multi-AZ
- Higher cost
- Higher availability
- Suitable for business-critical workloads

Idle Capacity vs Scaling Delays

Always On (Idle Capacity)
- Higher cost
- Immediate capacity
- No scaling delay
- Suitable for unpredictable traffic

Scale on Demand
- Lower cost
- Potential scaling delay
- Risk of temporary performance impact
- Suitable for predictable workloads

Senior Insight

The lowest-cost architecture is not necessarily the architecture with the lowest business risk.

Portfolio Deliverable

- Cost comparison.
- Terraform configurations.
- Documented architecture trade-offs.
- Monitoring and reporting setup.

Example Terraform Tags

```
tags = {
  "Environment" = "production"
  "Project"     = "veriqta-app"
  "Owner"       = "platform"
  "CostCenter"  = "0018"
  "ManagedBy"   = "terraform"
  "Application" = "web-api"
}
```

Verification

- Cost tracking is enabled and accurate.
- Infrastructure performance metrics are collected.
- Pull requests show estimated cost.
- Architecture trade-offs are documented.
- Overall solution meets cost, reliability, and performance requirements.

Production Incident: The Infrastructure Is Healthy, but AWS Spending Tripled

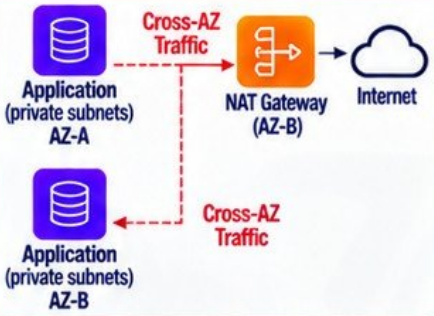
Failure scenario: An infrastructure change increases cross-AZ traffic and NAT gateway processing costs. Application health metrics remain normal.

Key Investigation Areas

1. AWS Cost Explorer and cost allocation data.
2. Cross-AZ data transfer.
3. NAT gateway processing volumes.
4. VPC Flow Logs.
5. Recent Terraform changes.
6. Whether network placement or routing caused the increase.

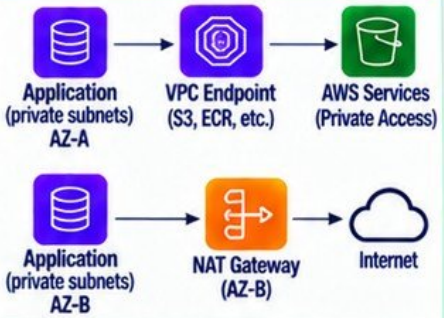
Architecture Comparison (Simplified)

Current Architecture (Higher Cost)



- Cross-AZ traffic between subnets
- All outbound traffic through NAT Gateway
- Increased NAT processing and data transfer costs

Optimized Architecture (Lower Cost)



- Use VPC endpoints for AWS services
- Keep traffic within the same AZ
- Reduce NAT gateway processing and cross-AZ costs

AWS Cost Explorer (Example)

Service	Before Change	After Change	Increase
NAT Gateway	$120.50	$1,020.30	+747%
Data Transfer (Cross-AZ)	$80.10	$420.60	+425%
EC2	$210.20	$215.30	+2%
RDS	$310.40	$312.10	+1%
Others	$95.30	$98.00	+3%
Total	**$816.50**	**$2,066.30**	**+153%**

VPC Flow Logs (Sample)

```
srcaddr     dstaddr       bytes     action
10.0.1.10   3.5.140.10    1048576   ACCEPT
10.0.2.15   3.5.140.10    2097152   ACCEPT
10.0.1.10   52.95.110.5   3145728   ACCEPT
10.0.2.15   52.95.110.5   2097152   ACCEPT
10.0.1.10   3.5.140.10    4194304   ACCEPT
10.0.2.15   3.5.140.10    2097152   ACCEPT
...
```

Terraform Change (Example)

```
# Changed subnet configuration
resource "aws_subnet" "app" {
  cidr_block        = "10.0.[illegible]/24"
  availability_zone = "us-east-1a"
}

resource "aws_route_table" "private" {
  route {
    cidr_block     = "0.0.0.0/0"
    nat_gateway_id = aws_nat_gateway.main.id
  }
}
```

Key Findings (Possible Causes)

- Cross-AZ traffic between resources in different Availability Zones.
- All outbound traffic routed through NAT gateway instead of VPC endpoints.
- Increased NAT gateway processing volumes.
- Terraform change modified subnet placement or routing configuration.

Cost Reduction Strategies

1. Use VPC endpoints for AWS services (S3, ECR, CloudWatch, etc.).
2. Keep resources in the same AZ to avoid cross-AZ data transfer.
3. Optimize NAT gateway usage (one per AZ, only when required).
4. Review and correct subnet routing.
5. Implement cost alerts and budgets.

Cost Impact (Example)



Senior Challenge

Reduce the unexpected cost without weakening security or availability.

Verification

- Measured costs are reduced.
- Application performance remains normal.
- Required network controls are preserved.
- Cost alerts are configured.

Portfolio Deliverable

- Cost incident analysis.
- Architectural correction.
- Terraform configurations.
- Measured financial impact.

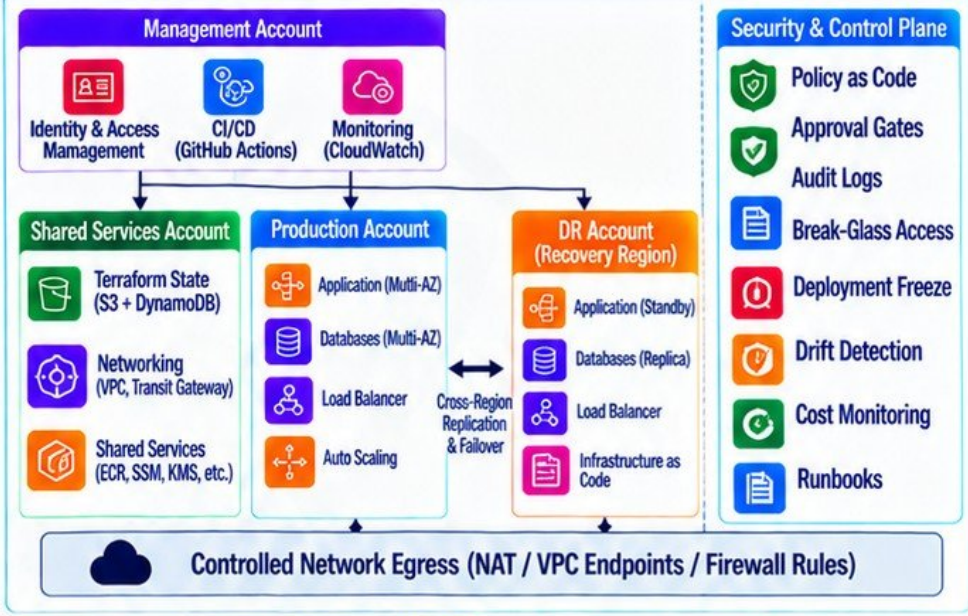
Build a Terraform Platform That Can Recover When Multiple Controls Fail

Project: Combine the strongest components from the previous nine projects into one production-style infrastructure platform.

Key Architecture Components

1. Multi-account AWS isolation.
2. Secure Terraform state backends.
3. Controlled network egress.
4. Highly available application infrastructure.
5. Versioned Terraform modules.
6. Policy-controlled CI/CD.
7. Drift detection.
8. Disaster recovery capability.
9. Cost and reliability monitoring.

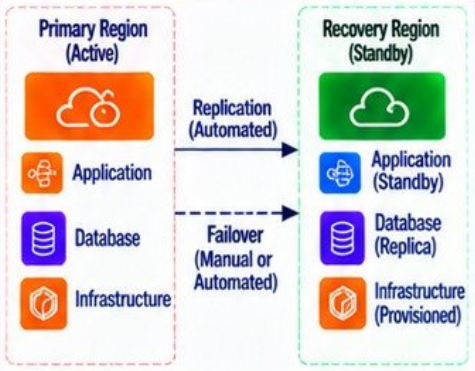
High-Level Architecture (Simplified)



Operational Control Plane

- Break-glass access.
- Deployment freeze mechanism.
- Emergency change reconciliation.
- Infrastructure ownership registry.
- Recovery runbooks.
- Auditable approval boundaries.

Disaster Recovery Flow (Simplified)



CI/CD Pipeline (Controlled)



Key Controls

- terraform fmt, validate, and plan
- Policy as code (e.g., OPA/Sentinel)
- Environment approvals
- Short-lived AWS credentials (OIDC)
- Restricted production apply roles
- Audit trails for all changes

Failure Domains (Example)

Shared Failure Domains (Must be resilient)	Independently Recoverable (Keep isolated)
• AWS Region	• Application services
• Identity provider	• Databases (replicated)
• DNS	• Terraform state (multi-region)
• Networking (core)	• CI/CD systems
• Secrets management	• Monitoring and logging

Senior Architecture Challenge

Identify which platform dependencies are shared failure domains and which must remain independently recoverable.

Verification

- Platform can recover when multiple controls fail.
- Security and compliance controls remain enforced.
- Infrastructure is reproducible from code.
- DR environment is activated and validated.
- Monitoring, cost, and reliability goals are met.

Portfolio Deliverable

- Complete architecture.
- Terraform repository.
- Decision records (ADRs).
- Security model.
- Operational runbooks.
- Recovery test results.

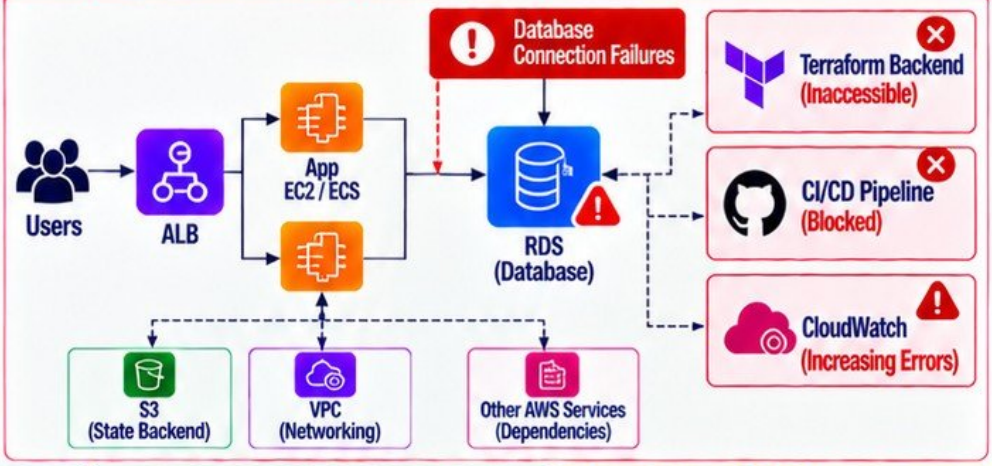
Final Production Exercise:
Recover AWS Without Trusting Terraform State

Compound incident scenario: A critical application experiences database connection failures shortly after an infrastructure deployment. An engineer attempts emergency remediation, but the Terraform backend is inaccessible. A previous module update has also introduced unexpected replacement plans.

Current Situation

1. Production pipeline is blocked.
2. Some infrastructure was modified manually.
3. The current Terraform state cannot be trusted.
4. CloudWatch reports increasing application errors.
5. The recovery environment has not been tested recently.

Simplified Architecture (Current State)



Your Mission

1. Establish customer impact and freeze unsafe infrastructure changes.
2. Identify the actual state of AWS resources independently of Terraform.
3. Investigate backend availability, state integrity, and module changes.
4. Restore safe operational access through approved emergency procedures.
5. Determine whether to recover the application, restore state, or activate disaster recovery.
6. Reconcile live infrastructure and Terraform configuration.
7. Validate security controls, application health, and data integrity.
8. Produce an incident report and prevention plan.

Investigation Workflow (Simplified)

- **Assess Impact** (Check application health, customer impact, and recent changes)
- **Investigate AWS** (Identify actual resource state independent of Terraform)
- **Check Terraform Backend** (Availability, state integrity, and module changes)
- **Recover Access / Stabilize** (Use approved emergency procedures)
- **Decide and Recover** (Recover application, restore state, or activate disaster recovery)
- **Reconcile and Validate** (Reconcile infrastructure, validate security, application health, and data integrity)

Key Tools and Evidence

- CloudWatch (logs, metrics, alarms)
- AWS CLI (inspect live resources)
- VPC Flow Logs (network analysis)
- RDS logs (database connectivity)
- Terraform plan (identify replacement changes)
- S3 backend (availability and access logs)
- Git history (recent module updates)
- IAM and CloudTrail (manual changes audit)

Recovery Decision Options

- Recover Application (Fix current environment)
- Restore State (Rebuild or repair Terraform state)
- Activate Disaster Recovery (Use recovery environment)

Final Senior Engineering Decision

- Which infrastructure changes are safe to automate?
- Which require human approval?
- Which should never occur during an active incident?

Verification

- Application is operational.
- Security controls are enforced.
- Terraform configuration is reconciled with live infrastructure.
- No unintended resource changes.
- Data integrity is verified.
- Incident is documented.

Portfolio Deliverable

Complete incident simulation containing:

- Architecture diagram.
- Terraform code.
- Diagnostic evidence.
- Recovery timeline.
- Verified outcome.
- Architecture decision record.